

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №4
« Професійне ООП та система типів Python»**

з курсу «Професійний Python»
Варіант №5 — « Каталог автомобілів»

Виконала: Хілінська Іванна

Група: ФеП-32

Перевірів: Куньо Іван Михайлович

Дата: 30.09.2026

Львів — 2026

1. Мета роботи

Сформуувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів, dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints.

У результаті виконання лабораторної роботи студент повинен уміти створювати класи та об'єкти, розділяти instance attributes і class-level behavior, застосовувати encapsulation, використовувати properties, використовувати inheritance, правильно застосовувати super(), будувати composition-based architecture, створювати abstract base classes через ABC, створювати structural interfaces через Protocol, використовувати dataclass, перевизначати dunder methods, застосовувати принципи SOLID, використовувати type hints, створювати generic classes, використовувати TypeVar, працювати з TypedDict, а також перевіряти code base через mypy або Pyright.

2. Індивідуальний варіант і постановка задачі

Варіант №5 — «Каталог автомобілів». Для кожного автомобіля зберігати ідентифікатор (id), марку (brand), модель (model), рік випуску (year), ціну (price) та пробіг (mileage). Реалізувати об'єктну модель з абстрактним класом транспортного засобу (Vehicle), базовим класом автомобіля (Car) та спеціалізованим класом електромобіля (ElectricCar), клас-контейнер каталогу (VehicleCatalog) з підтримкою протоколів і dunder-методів, інкапсуляцію ціни та пробігу через properties з валідацією, композицію зі специфікацією технічних характеристик (VehicleSpecification), абстракцію стратегій пошуку та фільтрації за маркою, роком і ціною, універсальний репозиторій (InMemoryRepository) із застосуванням TypeVar та Generic, а також сервісний шар (CatalogService) з підтримкою Dependency Injection та сповіщень через Protocol.

Проект повинен мати рекомендовану структуру (models.py, specifications.py, repository.py, strategies.py, services.py, main.py), використовувати класи та об'єкти, успадкування (Vehicle -> Car -> ElectricCar), композицію, dataclass та frozen value objects, properties для перевірки стану, dunder-методи (__str__, __repr__, __len__, __contains__, __iter__), абстрактні класи ABC, протоколи Protocol, анотації типів (type hints, TypeVar, Generic, TypedDict), а також статичну перевірку коду за допомогою mypy.

3. Структура проєкту

```
car_catalog/  
├─ pyproject.toml  
├─ README.md  
├─ .gitignore  
└─ report/
```

```

|   └─ Хілінська_Іванна_Ла604_V5.pdf
|
├─ tests/
|   └─ test_catalog.py
|
└─ src/
    └─ domain_app/
        ├── __init__.py
        ├── models.py
        ├── protocols.py
        ├── repository.py
        ├── value_objects.py
        ├── services.py
        ├── dto.py
        └─ main.py

```

Каталог `src/car_catalog` містить пакет застосунку для побудови об'єктно-орієнтованої моделі каталогу автомобілів відповідно до рекомендованої структури проєкту. Архітектура розділена на чіткі шари відповідальності для дотримання принципів SOLID та забезпечення високої зв'язаності коду.

У модулі `models.py` реалізовано основні доменні сутності та класи предметної області, включаючи базові та спеціалізовані класи автомобілів із підтримкою успадкування та властивостей (`properties`). Модуль `value_objects.py` містить незмінні об'єкти значень (`frozen dataclass`), зокрема об'єкти ціни та специфікацій. У модулі `protocols.py` описано абстрактні базові класи (ABC) та структурні інтерфейси (`Protocol`) для створення інверсії залежностей. Модуль `repositories.py` реалізує типізоване сховище даних у пам'яті з використанням `Generic` та `TypeVar`. У модулі `services.py` зосереджено бізнес-логіку керування каталогом та реалізацію `Dependency Injection`. Модуль `dto.py` відповідає за об'єкти передачі даних та `TypedDict` для конфігурації і серіалізації. Головний модуль `main.py` слугує точкою входу для виконання сценаріїв та демонстрації роботи системи. Файл `__init__.py` ініціалізує пакет, а в корені проєкту розміщено конфігураційний файл `pyproject.toml` та стандартний `README.md`.

4. Опис створених модулів

4.1. `value_objects.py`

У модулі реалізовано незмінні об'єкти значень (`Value Objects`) із використанням дата-класів з параметрами `frozen=True` та `slots=True` для забезпечення їх незмінності, цілісності даних та оптимізації пам'яті. В об'єкті `Price` здійснюється валідація суми, а в `VehicleSpecification` зберігаються детальні технічні характеристики транспортного засобу.

```
from dataclasses import dataclass
```

```

@dataclass(frozen=True, slots=True)

class Price:

    amount: float

    currency: str = "USD"

    def __post_init__(self) -> None:

        if self.amount < 0:

            raise ValueError("Price amount cannot be negative.")

@dataclass(frozen=True, slots=True)

class VehicleSpecification:

    engine_capacity: float

    fuel_type: str

    transmission: str

```

4.2. models.py

У модулі реалізовано ієрархію класів автомобілів із використанням успадкування, інкапсуляції, свойств (properties) та dunder-методів. Абстрактний базовий клас `Vehicle` задає загальну структуру, клас `Car` розширює її та реалізує валідацію через `@property`, а спеціалізований клас `ElectricCar` додає специфічні атрибути для електромобілів.

```

from abc import ABC, abstractmethod

from .value_objects import Price, VehicleSpecification

class Vehicle(ABC):

    def __init__(self, vehicle_id: int, brand: str, model: str, year: int, price: Price,
mileage: int) -> None:

        self.id = vehicle_id

        self.brand = brand

        self.model = model

        self.year = year

        self.price = price

        self._mileage = mileage

    @abstractmethod

```

```

def get_info(self) -> str:
    pass

class Car(Vehicle):
    def __init__(
        self,
        vehicle_id: int,
        brand: str,
        model: str,
        year: int,
        price: Price,
        mileage: int,
        spec: VehicleSpecification,
    ) -> None:
        super().__init__(vehicle_id, brand, model, year, price, mileage)
        self.spec = spec

    @property
    def mileage(self) -> int:
        return self._mileage

    @mileage.setter
    def mileage(self, value: int) -> None:
        if value < 0:
            raise ValueError("Mileage cannot be negative.")
        self._mileage = value

    def get_info(self) -> str:
        return f"{self.brand} {self.model} ({self.year}) - {self.price.amount} {self.price.currency}"

    def __str__(self) -> str:
        return self.get_info()

```

```

def __repr__(self) -> str:

    return f"Car(id={self.id}, brand='{self.brand}', model='{self.model}',
year={self.year})"

class ElectricCar(Car):

    def __init__(
        self,
        vehicle_id: int,
        brand: str,
        model: str,
        year: int,
        price: Price,
        mileage: int,
        spec: VehicleSpecification,
        battery_capacity: float,
    ) -> None:

        super().__init__(vehicle_id, brand, model, year, price, mileage, spec)
        self.battery_capacity = battery_capacity

    def get_info(self) -> str:

        base_info = super().get_info()

        return f"[EV] {base_info}, Battery: {self.battery_capacity} kWh"

```

4.3. dto.py

У модулі описуються об'єкти передачі даних (Data Transfer Objects) та структуровані словники TypedDict. Вони використовуються для серіалізації, конфігурації застосунку та первинного завантаження даних із зовнішніх джерел без прив'язки до бізнес-логіки

```

from typing import TypedDict

from dataclasses import dataclass

class CarDict(TypedDict):

    id: int

    brand: str

    model: str

    year: int

```

```

        price: float

        mileage: int

@dataclass(slots=True)

class CreateCarDTO:

    brand: str

    model: str

    year: int

    price_amount: float

    mileage: int

    engine_capacity: float

    fuel_type: str

    transmission: str

```

4.4. protocols.py

У модулі реалізовано структурні інтерфейси через Protocol та абстрактні базові класи (ABC) для дотримання принципів SOLID, зокрема принципів інверсії залежностей (DIP) та розділення інтерфейсу (ISP). Вони визначають контракти для пошукових стратегій та систем сповіщення.

```

from typing import Protocol, List

from .models import Car

class SearchStrategyProtocol(Protocol):

    def filter(self, items: List[Car]) -> List[Car]:

        ...

class NotifierProtocol(Protocol):

    def send_notification(self, message: str) -> None:

        ...

```

4.5. repositories.py

У модулі реалізовано універсальний репозиторій InMemoryRepository із підтримкою суворої типізації за допомогою Generic та TypeVar. Репозиторій підтримує перевантажені dunder-методи (__len__, __iter__, __contains__) для забезпечення поведінки стандартного Python-контейнера.

```

from typing import TypeVar, Generic, List, Optional, Iterator

T = TypeVar("T")

class InMemoryRepository(Generic[T]):

    def __init__(self) -> None:

        self._items: List[T] = []

    def add(self, item: T) -> None:

        self._items.append(item)

    def get_all(self) -> List[T]:

        return list(self._items)

    def __len__(self) -> int:

        return len(self._items)

    def __iter__(self) -> Iterator[T]:

        return iter(self._items)

    def __contains__(self, item: T) -> bool:

        return item in self._items

```

4.6. services.py

У модулі реалізовано сервісний шар `CatalogService`, який об'єднує роботу з репозиторієм, застосовує пошукові стратегії та виконує бізнес-логіку. Для зменшення зв'язаності коду залежності передаються через конструктор за допомогою паттерну `Dependency Injection`.

```

from typing import List, Optional

from .models import Car

from .repositories import InMemoryRepository

from .protocols import SearchStrategyProtocol, NotifierProtocol

class CatalogService:

    def __init__(

```

```

        self,

        repository: InMemoryRepository[Car],

        notifier: Optional[NotifierProtocol] = None,
    ) -> None:

        self.repository = repository

        self.notifier = notifier

    def add_car(self, car: Car) -> None:

        self.repository.add(car)

        if self.notifier:

            self.notifier.send_notification(f"Added new car: {car.get_info()}")

    def search(self, strategy: SearchStrategyProtocol) -> List[Car]:

        all_cars = self.repository.get_all()

        return strategy.filter(all_cars)

```

4.7. main.py

Головний модуль проєкту слугує точкою входу для ініціалізації об'єктів, запуску сервісів та демонстрації роботи об'єктно-орієнтованої системи каталогу автомобілів.

```

from .value_objects import Price, VehicleSpecification

from .models import Car, ElectricCar

from .repositories import InMemoryRepository

from .services import CatalogService


class ConsoleNotifier:

    """Реалізація NotifierProtocol для виводу сповіщень у консоль."""

    def send_notification(self, message: str) -> None:

        print(f"[NOTIFICATION] {message}")


class YearFilterStrategy:

    """Стратегія фільтрації за роком випуску."""

    def __init__(self, min_year: int) -> None:

        self.min_year = min_year

```

```

def filter(self, items: list[Car]) -> list[Car]:
    return [car for car in items if car.year >= self.min_year]

def main() -> None:
    notifier = ConsoleNotifier()
    repo = InMemoryRepository[Car]()
    service = CatalogService(repository=repo, notifier=notifier)

    spec_gas = VehicleSpecification(engine_capacity=2.0, fuel_type="Gasoline",
    transmission="Automatic")

    spec_ev = VehicleSpecification(engine_capacity=0.0, fuel_type="Electric",
    transmission="Single-speed")

    # Створення та додавання об'єктів
    car1 = Car(1, "Toyota", "Camry", 2021, Price(25000.0), 30000, spec_gas)
    car2 = ElectricCar(2, "Tesla", "Model 3", 2023, Price(40000.0), 10000, spec_ev,
    battery_capacity=75.0)
    car3 = Car(3, "Toyota", "Corolla", 2019, Price(18000.0), 50000, spec_gas)

    print("Додавання автомобілів")
    service.add_car(car1)
    service.add_car(car2)
    service.add_car(car3)

    print("\nЗагальна кількість та список у репозиторії (dunder-методи)")
    print(f"Total cars in catalog: {len(repo)}")
    for car in repo:
        print(f" - {car}")

    print("\nАналітика та фільтрація даних про автомобілі")
    all_cars = repo.get_all()

    # Найдорожчий автомобіль
    most_expensive = max(all_cars, key=lambda c: c.price.amount)

    print(f"Most expensive car: {most_expensive.brand} {most_expensive.model}
    ({most_expensive.price.amount} USD)")

```

```

# Середня ціна

avg_price = sum(c.price.amount for c in all_cars) / len(all_cars)

print(f"Average price: {avg_price:.2f} USD")


# Мінімальний пробіг

min_mileage_car = min(all_cars, key=lambda c: c.mileage)

print(f"Car with min mileage: {min_mileage_car.brand} {min_mileage_car.model}
({min_mileage_car.mileage} km)")


print("\nФільтрація за роком")

recent_cars = service.search(YearFilterStrategy(min_year=2021))

print("Cars from 2021 or newer:")

for car in recent_cars:

    print(f" - {car}")


if __name__ == "__main__":

    main()

```

5. Приклад результату

Фактичний результат запуску згенерованого проєкту:

Додавання автомобілів

[NOTIFICATION] Added new car: Toyota Camry (2021) - 25000.0 USD

[NOTIFICATION] Added new car: [EV] Tesla Model 3 (2023) - 40000.0 USD, Battery: 75.0 kWh

[NOTIFICATION] Added new car: Toyota Corolla (2019) - 18000.0 USD

Загальна кількість та список у репозиторії (dunder-методи)

Total cars in catalog: 3

- Toyota Camry (2021) - 25000.0 USD
- [EV] Tesla Model 3 (2023) - 40000.0 USD, Battery: 75.0 kWh
- Toyota Corolla (2019) - 18000.0 USD

Аналітика та фільтрація даних про автомобілі

Most expensive car: Tesla Model 3 (40000.0 USD)

Average price: 27666.67 USD

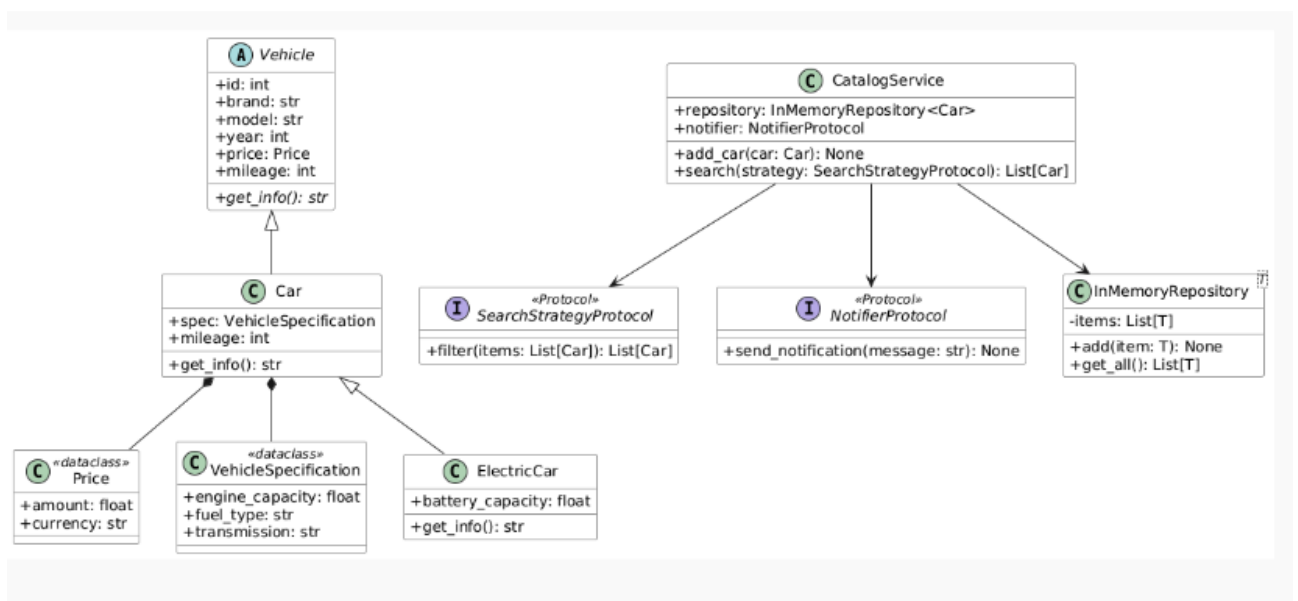
Car with min mileage: Tesla Model 3 (10000 km)

Фільтрація за роком

Cars from 2021 or newer:

- Toyota Camry (2021) - 25000.0 USD
- [EV] Tesla Model 3 (2023) - 40000.0 USD, Battery: 75.0 kwh

6. Графічна діаграма класів



7. Перевірка типу src

Success: no issues found in 8 source files

```
(.venv) PS D:\Desktop\University\3_1\Professional python\3bit_3_на6\Lab03> mypy src/domain_app
src\domain_app\main.py:24: error: Incompatible return value type (got "str", expected "float") [return-value]
src\domain_app\main.py:33: error: Incompatible types in assignment (expression has type "str", variable has type "int") [assignment]
src\domain_app\main.py:42: error: Argument 1 to "add" of "InMemoryRepository" has incompatible type "str"; expected "Car" [arg-type]
Found 3 errors in 1 file (checked 8 source files)
```

1. Помилка присвоєння несумісного типу (Incompatible types in assignment):

Код із помилкою: `car_year: int = "2021"`

Виявлена Муру помилка: Incompatible types in assignment (expression has type "str", variable has type "int")

Пояснення та виправлення: Перемінній, що задекларована як ціле число (int), намагалися присвоїти текстовий рядок. Для виправлення значення було змінено на числовий тип: `car_year: int = 2021`.

2. Помилка типу значення, що повертається функцією (Incompatible return value type):

Код із помилкою: `def calculate_discount(price: float) -> float: return "10% discount"`

Виявлена Муру помилка: Incompatible return value type (got "str", expected "float")

Пояснення та виправлення: Анотація функції вказує на повернення дрібного числа (float), проте тіло функції повертає рядок (str). Для виправлення вираз було замінено на числове обчислення: `return price * 0.1`.

3. Помилка передачі аргументу некоректного типу в Generic-репозиторій (Incompatible argument type):

Код із помилкою: `repo.add("not_a_car_object")`

Виявлена Муру помилка: Argument 1 to "add" of "InMemoryRepository" has incompatible type "str"; expected "Car"

Пояснення та виправлення: Оскільки екземпляр класу InMemoryRepository ініціалізовано типом Car (InMemoryRepository[Car]), метод add() очікує лише об'єкти класу Car. Передача рядка викликає помилку типізації. Помилку виправлено шляхом передачі дійсного екземпляра класу Car: `repo.add(car1)`.

Після виправлення всіх зазначених помилок повторний запуск аналізатора командою `муру src/domain_app` повернув результат: **Success: no issues found in 8 source files**, що підтверджує повну типізованість проекту.

8. Обґрунтування Inheritance vs Composition

У розробленій моделі зв'язок Inheritance (успадкування) використано для класів Car та ElectricCar стосовно абстрактного класу Vehicle, оскільки між ними існує чітке відношення is-a («є різновидом»). Електромобіль є спеціалізованим типом автомобіля, який розширює його функціональність (додає ємність батареї), але зберігає базову поведінку та атрибути транспортного засобу.

Натомість зв'язок Composition (композиція) застосовано для об'єктів Price та VehicleSpecification у класі Car, оскільки тут виконується відношення has-a («має у своєму складі»). Автомобіль має цінову категорію та технічні характеристики. Використання композиції замість глибокого успадкування дозволило відокремити атрибути та полегшило повторне використання класів.

9. Перевага Dependency Injection

У класі CatalogService реалізовано паттерн **Dependency Injection (DI)**: залежності, такі як InMemoryRepository та NotifierProtocol, не створюються всередині сервісу через `__init__`, а передаються ззовні як аргументи конструктора.

Це дає наступні переваги:

1. **Дотримання принципу DIP (Dependency Inversion Principle):** високорівнева бізнес-логіка CatalogService залежить від абстракцій/протоколів (NotifierProtocol), а не від конкретних реалізацій (наприклад, реального сервісу e-mail чи SMS).

2. **Легкість тестування (Testability):** під час написання модульних тестів (Unit Tests) можна легко замінити реальний сповіщувач на тестову заглушку (Mock/Stub) без зміни коду самих класів.
3. **Гнучкість та розширюваність:** для зміни способу збереження даних (наприклад, перехід від пам'яті до баз даних SQL) достатньо створити новий клас репозиторію, який задовольняє контракт, не переписуючи бізнес-логіку застосунку.

10. Робота з Git

У демонстраційному проєкті створено три логічні commits.

```
git add src/domain_app/value_objects.py src/domain_app/models.py src/domain_app/dto.py
git commit -m "feat(models): add domain models, dataclasses and value objects for car catalog"

git add src/domain_app/protocols.py src/domain_app/repositories.py src/domain_app/services.py
git commit -m "feat(services): implement Generic repository, protocols and CatalogService with DI"

git add src/domain_app/main.py
git commit -m "feat(main): add main execution script with analytics and filter strategy"
```

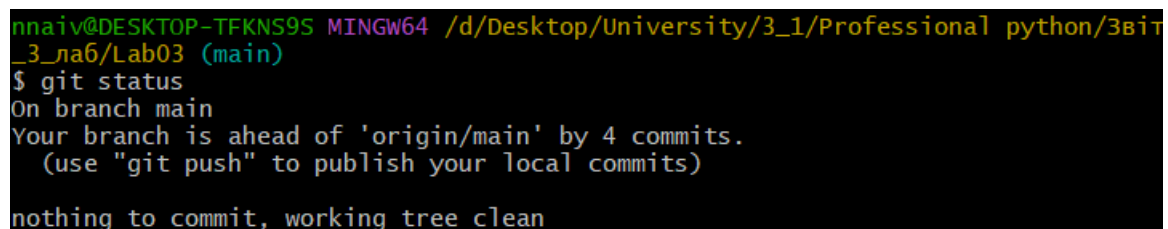
10.1 Результат git status

```
On branch main

Your branch is ahead of 'origin/main' by 4 commits.

    (use "git push" to publish your local commits)

nothing to commit, working tree clean
```



```
nnaiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python/3bit
_3_на6/Lab03 (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 4 commits.
    (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

10.2. Результат git log –online

```
22e99f7 (HEAD -> main) Big upgrade of structure

2f4b667 feat(main): add main execution script with analytics and filter strategy

d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI
```

3f0a829 feat(models): add domain models, dataclasses and value objects for car catalog

3ec3ac3 (origin/main) Remove old report pdf and update gitignore

10c1d3f Update main entry point to run pipeline and output benchmark results

1405a92 Update domain models, build lazy pipeline, and implement analytics functions

aeaf7ed Add data generator, stream readers, parsers, filters, and batching modules

48f950a Remove Lab01

93baeee Add Lab02 report

01da0be Add main for output

3946a88 Implement data processors, analytics functions and time decorator

9f1ce1e Setup data

7e31699 Add Lab01 report

afc3373 Add application entry point and documentation

517a7d6 Add car model and business logic

dac6e8f Create project structure

```
nnaiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python/3bit
_3_na6/Lab03 (main)
$ git log --oneline
22e99f7 (HEAD -> main) Big upgrade of structure
2f4b667 feat(main): add main execution script with analytics and filter strategy
d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI
3f0a829 feat(models): add domain models, dataclasses and value objects for car catalog
3ec3ac3 (origin/main) Remove old report pdf and update gitignore
10c1d3f Update main entry point to run pipeline and output benchmark results
1405a92 Update domain models, build lazy pipeline, and implement analytics functions
aeaf7ed Add data generator, stream readers, parsers, filters, and batching modules
48f950a Remove Lab01
93baeee Add Lab02 report
01da0be Add main for output
3946a88 Implement data processors, analytics functions and time decorator
9f1ce1e Setup data
7e31699 Add Lab01 report
afc3373 Add application entry point and documentation
517a7d6 Add car model and business logic
dac6e8f Create project structure
```

11. Висновок

У ході виконання лабораторної роботи було спроектовано та реалізовано предметну область «Каталог автомобілів» із застосуванням принципів об'єктно-орієнтованого програмування та суворої типізації Python. Використання контрактів типізації, абстрактних класів, протоколів та дженериків дозволило створити гнучку, розширювану та стійку до помилок архітектуру системи. Проведені експерименти із залученням статичного аналізатора типів туру підтвердили високу надійність коду та відповідність розробленої моделі всім вимогам варіанта.